AXI + SPI/I2C Protocol

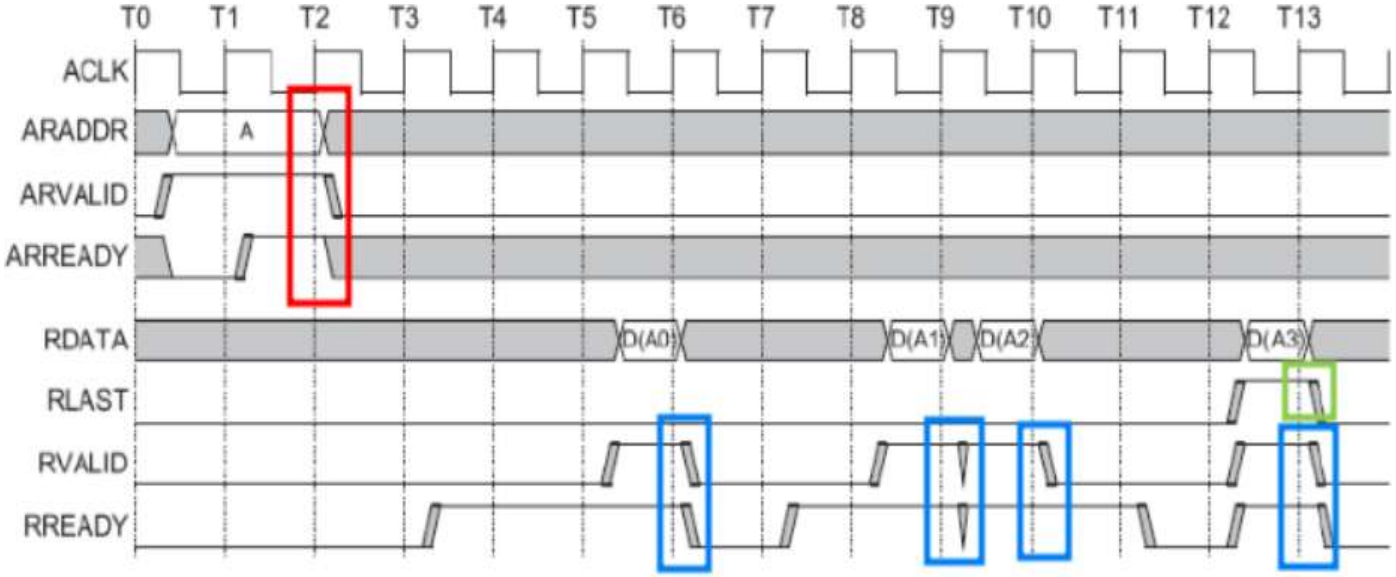
최수영

2026. 05. 08

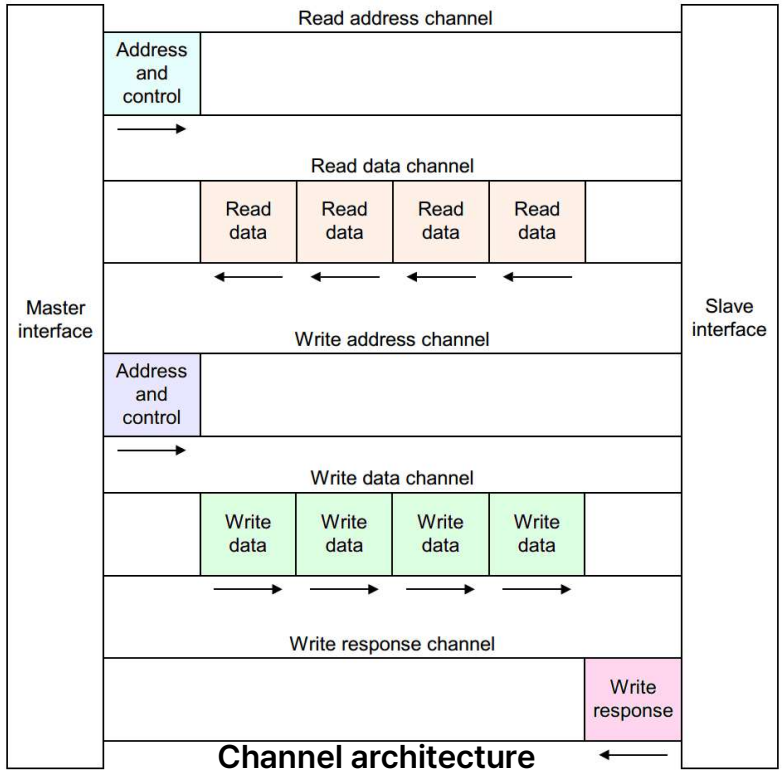
Contents

AXI Protocol Concept

- **A**dvanced e**X**tensible **I**nterface protocol
- **High-performance**, **point-to-point** communication standard
- **5 independent channels**, **burst** data transfer, **handshaking**
- AXI4, AXI4-Lite, AXI4-Stream



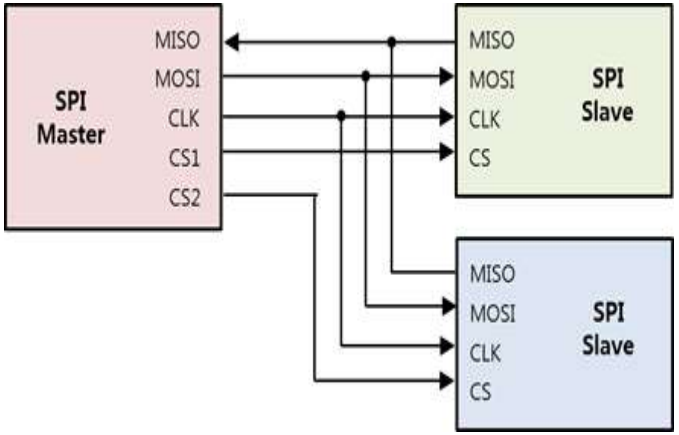
AXI transfer in read address / read data channel

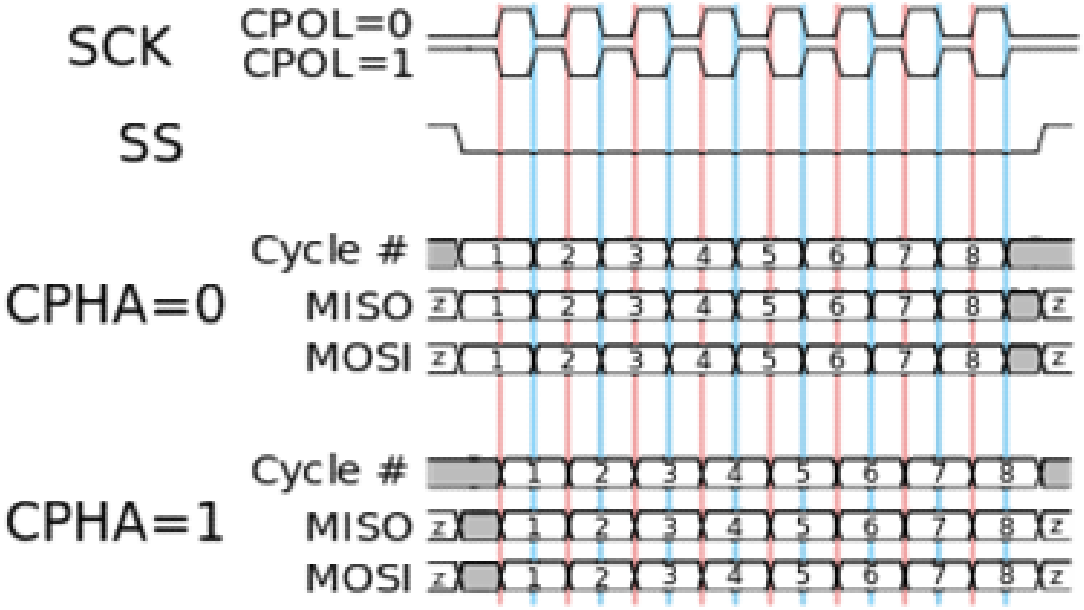


Channel architecture

SPI Protocol Concept

- **S**erial **P**eripheral **I**nterface Protocol
- **Synchronous**, **full-duplex**, **4-wire** serial communication
- **Short-distance**, **high-speed** data exchange

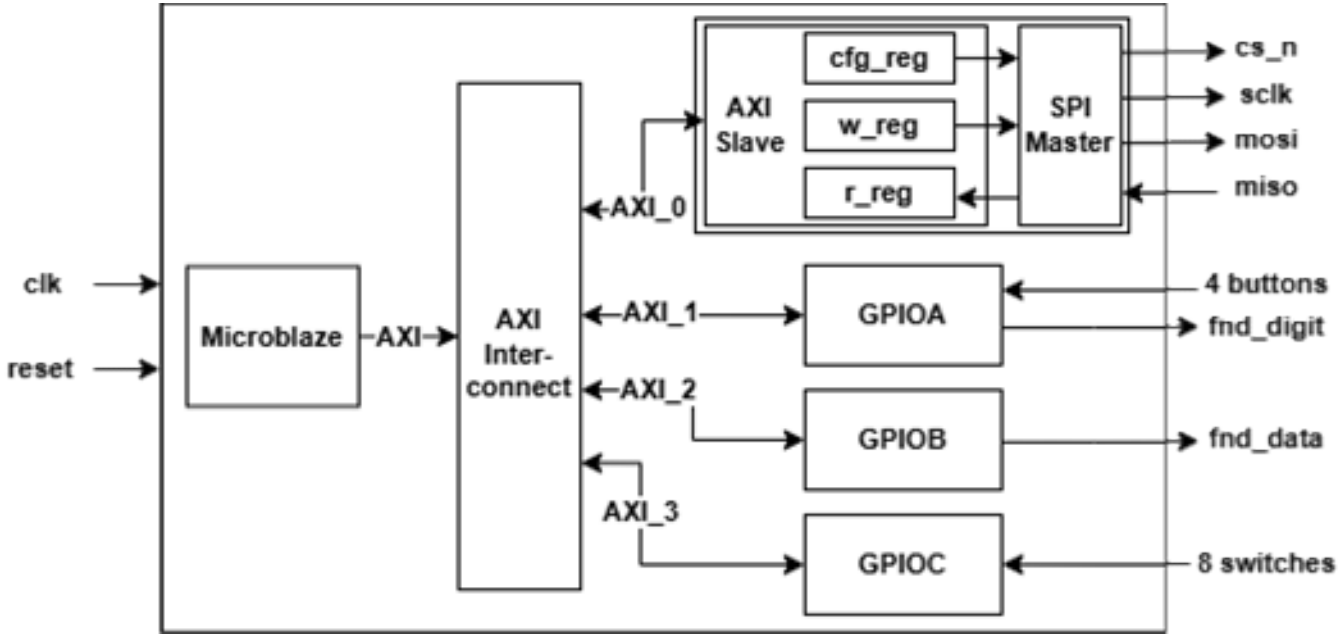




Mode		Sampling Timing
CPOL	CPHA	
0	0	Rising edge
0	1	Falling edge
1	0	Falling edge
1	1	Rising edge

AXI + SPI Master Design

- C program → Microblaze CPU → AXI data transfer → SPI Master module
- GPIO
  - 4 buttons
  - 7-segment
  - 8 switches



cfg_reg

31	...	15	14	13	12	11	10	9	8	...	1	0
Res.		clk_div								Res.	cpha	cpol

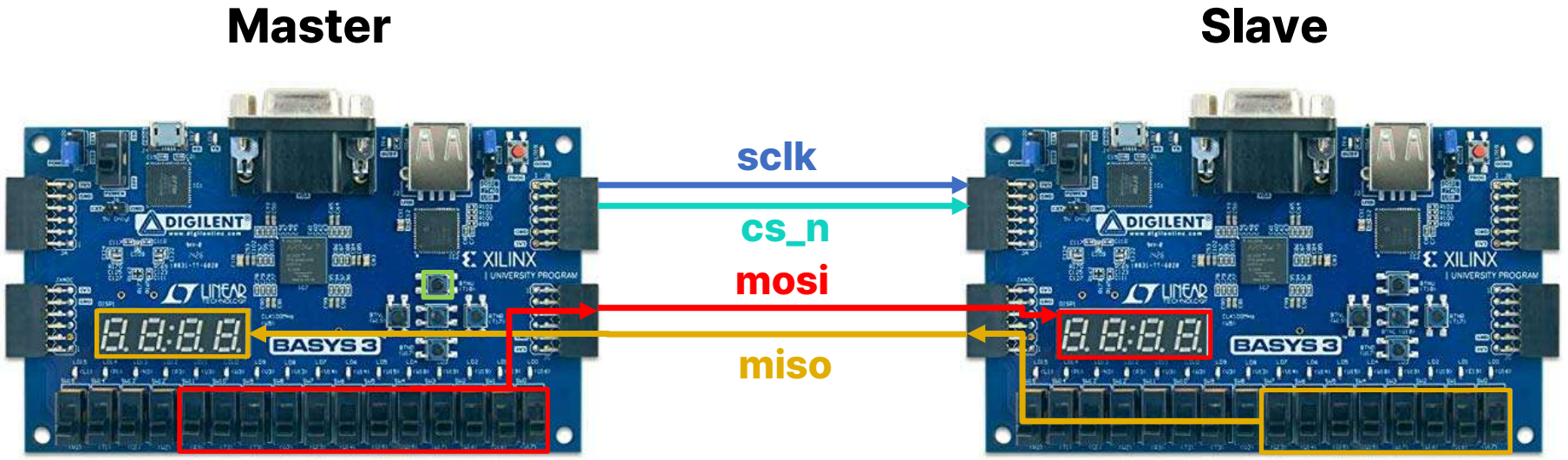
w_reg

31	...	7	6	5	4	3	2	1	0
start	Res.	tx_data							

r_reg

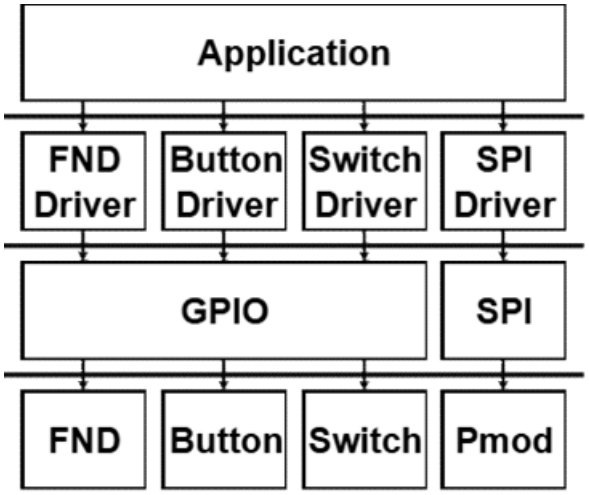
31	...	9	8	7	6	5	4	3	2	1	0
Res.		busy	done	rx_data							

AXI + SPI Implementation

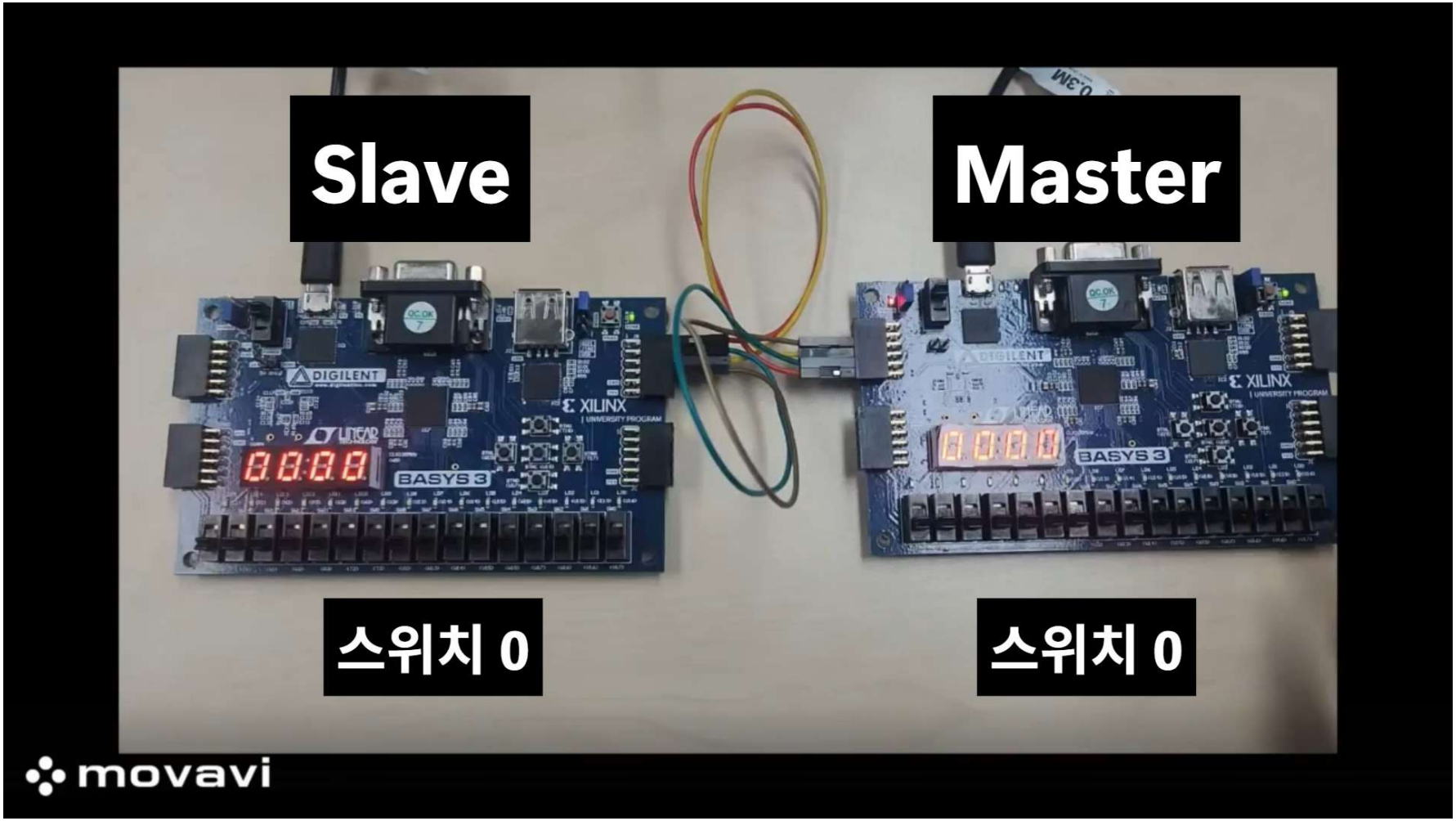


- Start SPI data transfer with Master U button
- Master 8 switches → SPI mosi → Slave FND (0-255)

  Slave 8 switches → SPI miso → Master FND (0-255)
- Simultaneous SPI READ/WRITE



AXI + SPI Implementation Demo



AXI + SPI Master Verification Scenario

- Sequence 생성 횟수: 1000
- Master → Slave (MOSI), Slave → Master (MISO) 동시 데이터 송수신
- 송신 데이터(tx_data) 8비트 랜덤 생성
- AXI protocol을 통해 SPI Master 제어
- Scoreboard PASS/FAIL 판단 기준
  - (Master/Slave tx_data) == (Slave/Master rx_data)
- Coverage
  - Master/Slave tx_data
    - 특이한 패턴을 가진 특정 값: 0x00, 0xff, 0x55, 0xaa, 0x01, 0x80
    - 값 범위 별로 구분: 0x00-0x0f, 0x10-0x1f, 0x20-0x2f, ... , 0xf0-0xff

AXI + SPI Master UVM Verification

Scoreboard Checker

```
[axi_spi_seq] seq send : [SPI Master] tx_data=0x64, [SPI Slave] tx_data=0x77
scoreboard] [PASS] M->S : 0x64, S->M : 0x77
[axi_spi_seq] seq send : [SPI Master] tx_data=0x27, [SPI Slave] tx_data=0xbb
scoreboard] [PASS] M->S : 0x27, S->M : 0xbb
[axi_spi_seq] seq send : [SPI Master] tx_data=0x1b, [SPI Slave] tx_data=0x36
scoreboard] [PASS] M->S : 0x1b, S->M : 0x36
```

Coverage

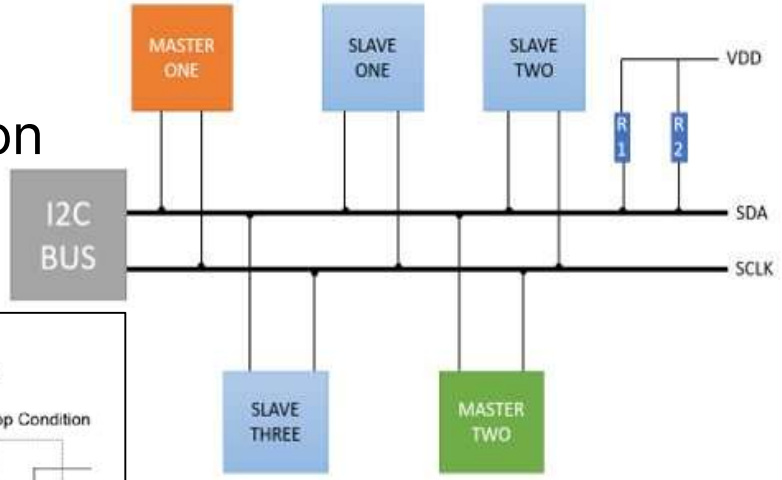
```
[axi_spi_coverage] coverage cg_data=100.0%
[axi_spi_coverage] coverage spi_m_tx_data=100.0%
[axi_spi_coverage] coverage spi_s_tx_data=100.0%
```

Scoreboard PASS/FAIL Summary

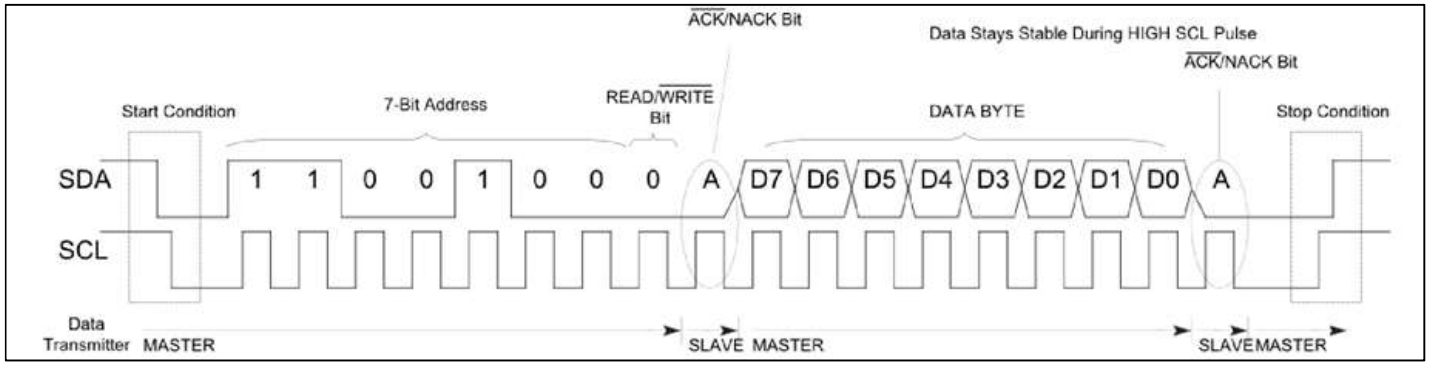
```
[axi_spi_scoreboard] pass_cnt = 1000/1000
[axi_spi_scoreboard] fail_cnt = 0/1000
```

I2C Protocol Concept

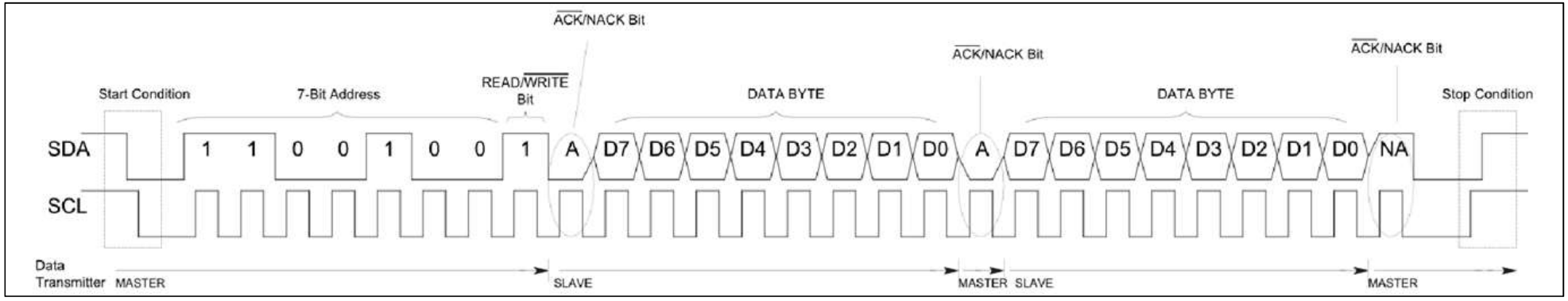
- **I**nter-**I**ntegrated **C**ircuit Protocol
- **Syncronous**, **half-duplex**, **2-wire** serial communication
- **Short-distance**, **multi-master/multi-slave**



WRITE

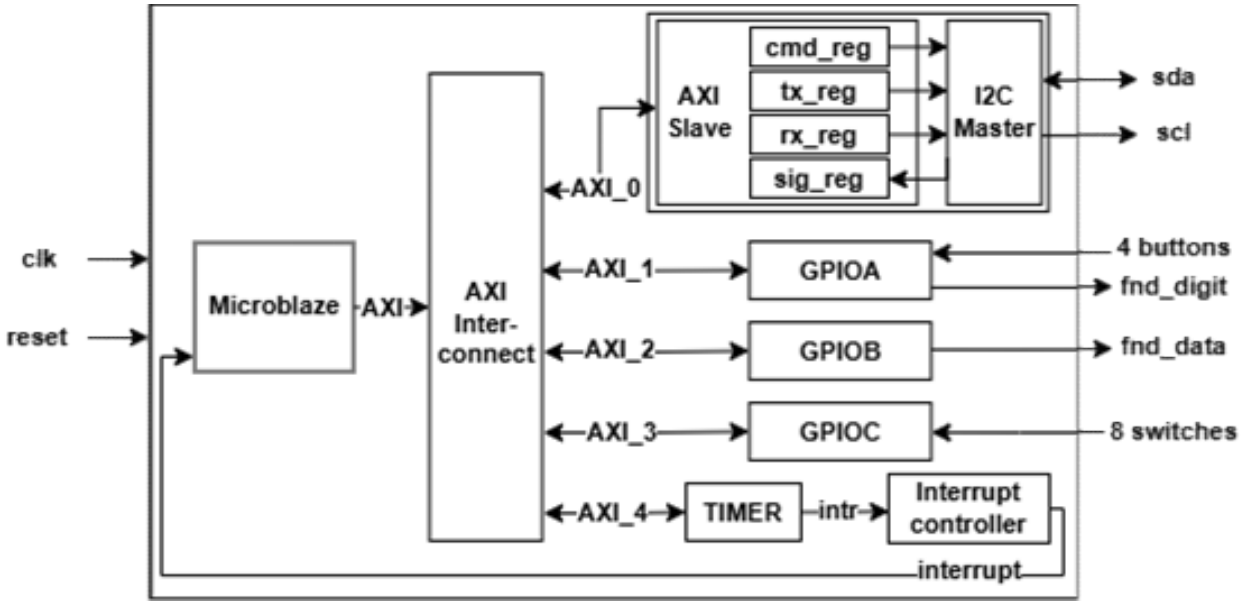


READ



AXI + I2C Master Design

- C program → Microblaze CPU → AXI data transfer → I2C Master module
- GPIO
  - 4 buttons
  - 7-segment
  - 8 switches
- Interrupt generation (10us) → 10usec upcounter increment



cmd_reg

31	...	5	4	3	2	1	0
Res.			ack_m	stop	read	write	start

tx_reg

31	...	8	7	6	5	4	3	2	1	0
Res.			tx_data							

rx_reg

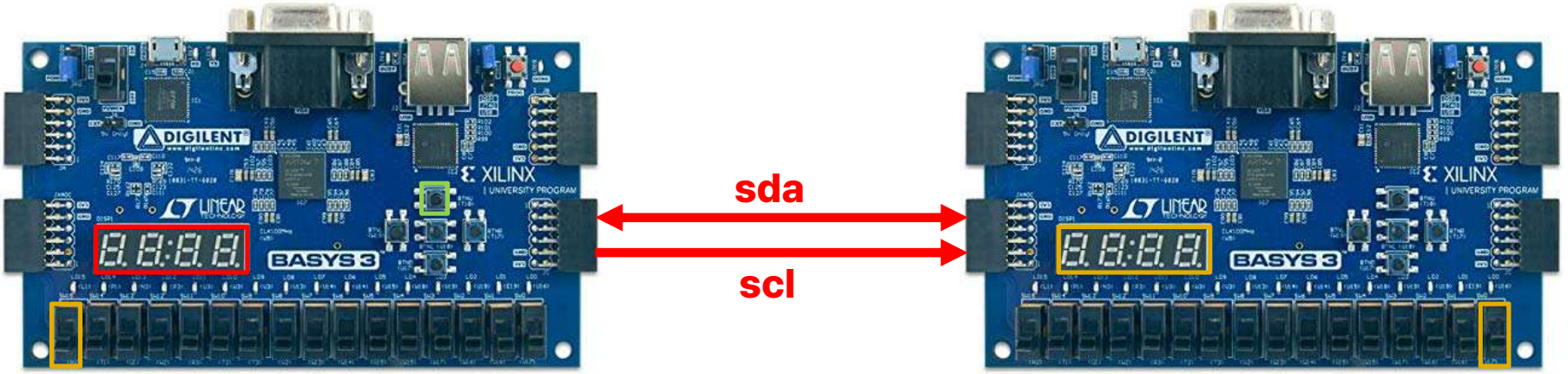
31	...	8	7	6	5	4	3	2	1	0
Res.			rx_data							

sig_reg

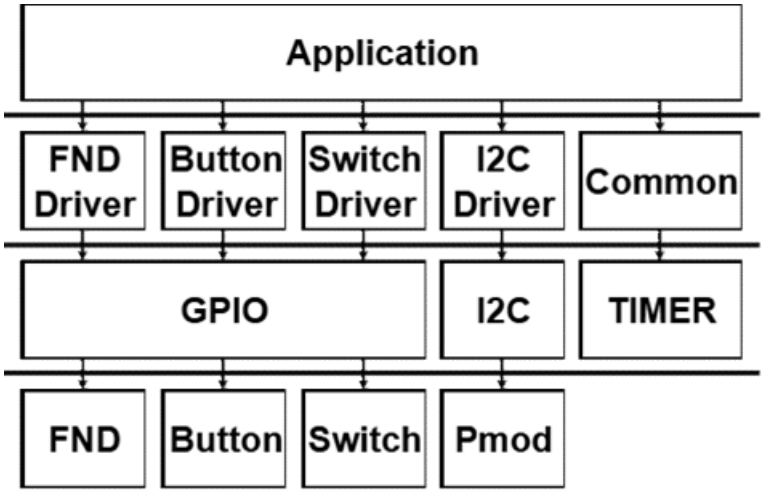
31	...	3	2	1	0
Res.			ack_s	done	busy

AXI + I2C Implementation

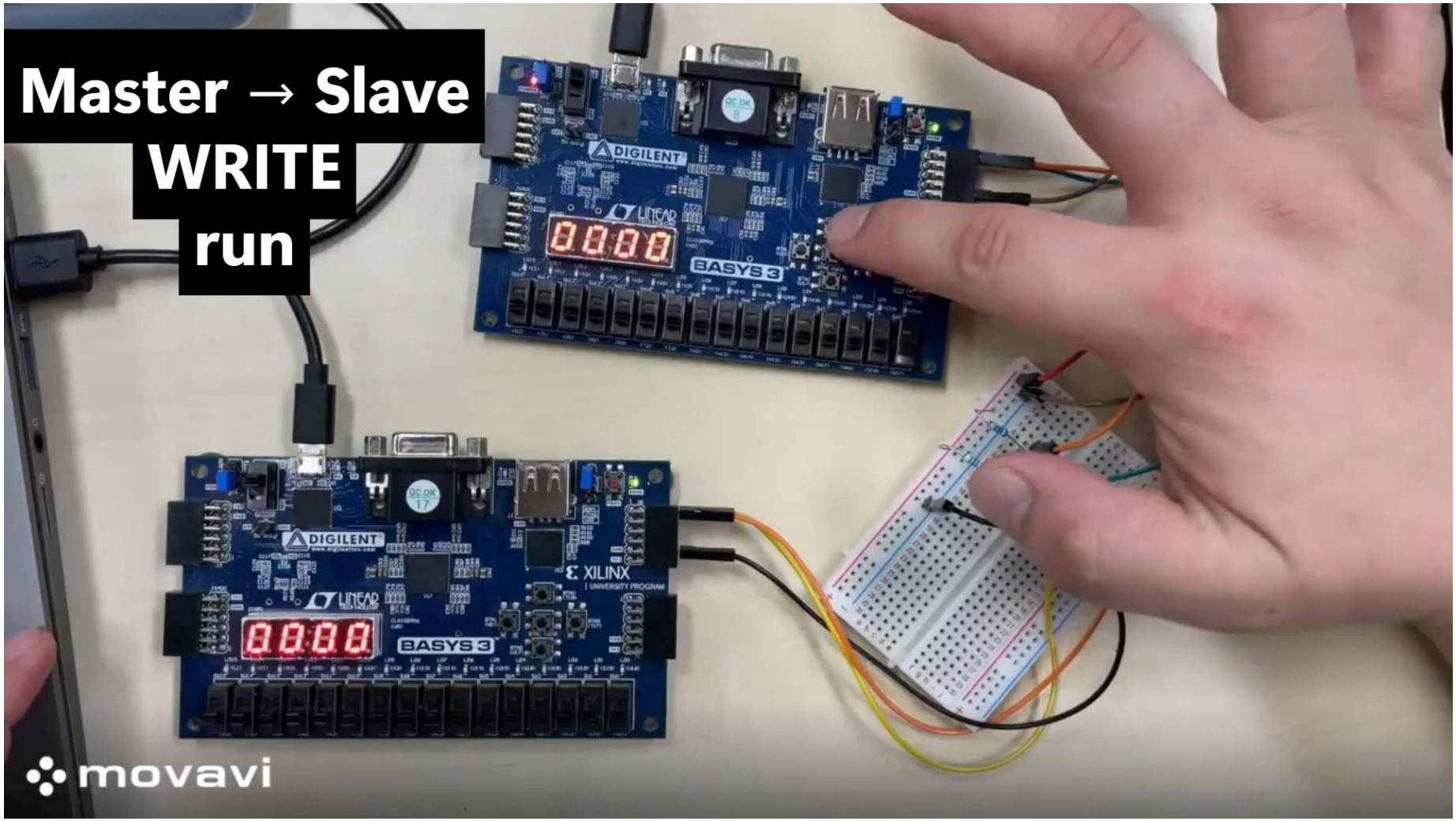
Master **Slave**



- Run/stop counter with Master U button
- Toggle WRITE ↔ READ mode with Master switch[15]
- Enable/disable slave counter with Slave switch[0]
  - WRITE mode
    - Master counter → Master/Slave FND
  - READ mode
    - Slave counter → Master/Slave FND



AXI + I2C Implementation Demo



AXI + I2C Master Verification Scenario

- Sequence 생성 횟수: 2000
- Master → Slave (WRITE) 또는 Slave → Master (READ)
- WRITE/READ, 송신 데이터(8-bit tx_data), 연속 데이터 송수신 횟수(1-16회) 랜덤 생성
- AXI protocol을 통해 I2C Master 제어
- Scoreboard PASS/FAIL 판단 기준
  - WRITE 시 (Master tx_data) == (Slave rx_data)
  - READ 시 (Slave tx_data) == (Master rx_data)
- Coverage
  - Master/Slave tx_data
    - 특이한 패턴을 가진 특정 값: 0x00, 0xff, 0x55, 0xaa, 0x01, 0x80
    - 값 범위 별로 구분: 0x00-0x0f, 0x10-0x1f, 0x20-0x2f, ... , 0xf0-0xff
  - Num_tr (연속 데이터 송수신 횟수)
    - 값 범위 별로 구분 : 1-8, 9-16

AXI + I2C Master UVM Verification

Scoreboard Checker (READ)

```
sqr@@seq [axi_i2c_seq] seq send : RW=1/0, num_tr=3, [Master] tx_data=0x8e, [Slave] tx_data=0x15
[axi_i2c_scoreboard] [PASS] S->M : 0x15
[axi_i2c_scoreboard] [PASS] S->M : 0x15
[axi_i2c_scoreboard] [PASS] S->M : 0x15
```

Scoreboard Checker (WRITE)

```
sqr@@seq [axi_i2c_seq] seq send : RW=0/1, num_tr=7, [Master] tx_data=0xd8, [Slave] tx_data=0x7f
[axi_i2c_scoreboard] [PASS] M->S : 0xd8
[axi_i2c_scoreboard] [PASS] M->S : 0xd8
[axi_i2c_scoreboard] [PASS] M->S : 0xd8
[axi_i2c_scoreboard] [PASS] M->S : 0xd8
[axi_i2c_scoreboard] [PASS] M->S : 0xd8
[axi_i2c_scoreboard] [PASS] M->S : 0xd8
[axi_i2c_scoreboard] [PASS] M->S : 0xd8
```

Coverage

```
[axi_i2c_coverage] coverage cg_data=100.0%
[axi_i2c_coverage] coverage i2c_m_tx_data=100.0%
[axi_i2c_coverage] coverage i2c_s_tx_data=100.0%
[axi_i2c_coverage] coverage num_tr=100.0%
[axi_i2c_coverage]  ===== Coverage Report =====
```

Scoreboard PASS/FAIL Summary

```
[axi_i2c_scoreboard] pass_cnt = 17260/17260
[axi_i2c_scoreboard] fail_cnt = 0/17260
```

Trouble Shooting – AXI WSTRB

```
gt.sqr@@seq [axi_spi_seq] seq send : spi_m_tx_data=0x27, spi_s_tx_data=0x44
gt.drv [axi_aw] start
gt.drv [axi_w] start
[axi_awready_gen] awready 1
[axi_wready_gen] wready 1
[[AXI_REG]] axi_awaddr=0x0, [3:2]=0x0, AXI_WDATA=0x400
            reg0:0, reg1:0, reg2:0, reg3:0
```

```
[axi_spi_monitor] axi_rdata=0
[axi_spi_monitor] monitor read : spi_m_tx_data=0x00, spi_s_tx_data=0x44
i_spi_scoreboard] [FAIL] M->S : 0x0 -> 0x0, S->M : 0x44 -> 0x0
```

```
virtual task run_phase(uvm_phase phase);
    // drive item to interface
    axi_spi_seq_item item;
    wait(v_if.rstn==1);

    v_if.wstrb = 4'hf;

    forever begin
        seq_item_port.get_next_item(item);

        wait(!v_if.drv_cb.spi_s_busy);
        // INIT
        axi_write(4'h0, ctrl_reg);
```

```
agt.sqr@@seq [axi_spi_seq] seq send : spi_m_tx_data=0x27
agt.drv [axi_aw] start
agt.drv [axi_w] start
 [axi_awready_gen] awready 1
 [axi_wready_gen] wready 1
 [[AXI_REG]] axi_awaddr=0x0, [3:2]=0x0, AXI_WDATA=0x400
             reg0:400, reg1:0, reg2:0, reg3:0
```

Conclusion

AXI 인터페이스를 기반으로 SPI와 I2C 프로토콜의 Master/SIave 모듈을 직접 설계하고 통합 검증하며 하드웨어 시스템의 데이터 흐름을 더 잘 이해할 수 있었습니다. 특히 UVM 환경에서 타이밍 이슈를 해결하며 설계의 완성도와 검증의 중요성을 깨달았습니다.